Chapter 6. Subwording and Transformers

Machine Translation: Advanced Topics

Vincent Vandeghinste

KU Leuven – Brussels

2026

Subwording in Neural Machine Translation

Why subwording? (word-level weaknesses)

Early NMT often used **word-level vocabularies**, but this creates problems:

- **OOV problem**: unseen words become `<unk>` (names, terminology, inflections, compounds).
- **Data sparsity**: related forms are independent symbols (*run*, *runs*, *running*, *ran*).
- **Large vocabularies** increase model size and training cost (bigger embeddings + output layers).

Subwording addresses all three simultaneously.

Core idea of subwording

Instead of treating the **word** as the smallest unit, we use **subword units** learned from data.

- More flexible than words: can compose unseen forms.
- More structured than characters: sequences stay manageable.

Example segmentation:

un + believ + able

Benefits:

- smaller vocabulary,
- no <unk> (everything can be composed),
- morphology is learned implicitly via recurring pieces.

Byte Pair Encoding (BPE): idea

BPE starts from **characters** and repeatedly merges the most frequent adjacent pair:

1. find most frequent adjacent symbol pair,
2. merge into a new symbol,
3. repeat for a fixed number of merge operations.

Resulting vocabulary contains:

- characters,
- frequent character sequences,
- full words where appropriate.

BPE: tiny illustrative segmentation

You can think of BPE as gradually building longer units:

$$u\ n\ b\ e\ l\ i\ e\ v\ a\ b\ l\ e \Rightarrow un + believ + able$$

In practice, merges are learned from corpus frequency statistics.

BPE: Reminder of the algorithm

Starting point: corpus split into characters + end-of-word symbol `</w>`.

Iterate:

1. Count all adjacent symbol pairs.
2. Select the most frequent pair.
3. Merge it into a new symbol.
4. Update corpus and repeat.

We stop after a fixed number of merges.

BPE Example

First we have a corpus.

> *Resumption of the session. You have requested a debate on this subject in the course of the next few days, during this part-session.*

Tokenization and frequencies

We word-tokenize the corpus and add spaces between all characters.
The original spaces are presented as `</w>`.
We count the occurrence of each word.

Tokenised word	Freq.	Tokenised word	Freq.
R e s u m p t i o n </w>	1	s u b j e c t </w>	1
o f </w>	2	i n </w>	1
t h e </w>	3	c o u r s e </w>	1
s e s s i o n </w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e </w>	1	, </w>	1
r e q u e s t e d </w>	1	d u r i n g </w>	1
a </w>	1	p a r t - s e s s i o n </w>	1
d e b a t e </w>	1		
o n </w>	1		
t h i s </w>	2		

Iteration 1

Most frequent adjacent pairs

Pair	Freq.
(e, </w>)	6
(n, </w>)	5
(t, h)	5
(e, s)	4
(o, n)	4
(i, o)	3

Selected merge: (e, </w>) → e</w>

Iteration 1: updated tokens

Tokenized word	Freq	Tokenized word	Freq
R e s u m p t i o n </w>	1	s u b j e c t </w>	1
o f </w>	2	i n </w>	1
t h e</w>	3	c o u r s e</w>	1
s e s s i o n </w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e</w>	1	, </w>	1
r e q u e s t e d </w>	1	d u r i n g </w>	1
a </w>	1	p a r t - s e s s i o n </w>	1
d e b a t e</w>	1		
o n </w>	1		
t h i s </w>	2		

Iteration 2

Most frequent adjacent pairs

Pair	Freq.
(n, </w>)	5
(t, h)	5
(e, s)	4
(o, n)	4
(i, o)	3
(h, e</w>)	3

Selected merge: (n, </w>) → n</w>

Iteration 2: updated tokens

Tokenized word	Freq	Tokenized word	Freq
R e s u m p t i o n</w>	1	s u b j e c t </w>	1
o f </w>	2	i n</w>	1
t h e</w>	3	c o u r s e</w>	1
s e s s i o n</w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e</w>	1	, </w>	1
r e q u e s t e d </w>	1	d u r i n g </w>	1
a </w>	1	p a r t - s e s s i o n</w>	1
d e b a t e</w>	1		
o n</w>	1		
t h i s </w>	2		

Iteration 3

Most frequent adjacent pairs

Pair	Freq.
(t, h)	5
(e, s)	4
(o, n</w>)	4
(i, o)	3
(h, e</w>)	3
(s, </w>)	3

Selected merge: (t, h) $\rightarrow$ th

Iteration 3: updated tokens

Tokenized word	Freq	Tokenized word	Freq
R e s u m p t i o n</w>	1	s u b j e c t </w>	1
o f </w>	2	i n</w>	1
th e</w>	3	c o u r s e</w>	1
s e s s i o n</w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e</w>	1	, </w>	1
r e q u e s t e d </w>	1	d u r i n g </w>	1
a </w>	1	p a r t - s e s s i o n</w>	1
d e b a t e</w>	1		
o n</w>	1		
th i s </w>	2		

Iteration 4

Most frequent adjacent pairs

Pair	Freq.
(e, s)	4
(o, n</w>)	4
(i, o)	3
(th, e</w>)	3
(s, </w>)	3
(s, u)	2

Selected merge: (e, s) $\rightarrow$ es

Iteration 4: updated tokens

Tokenized word	Freq	Tokenized word	Freq
R es u m p t i o n</w>	1	s u b j e c t </w>	1
o f </w>	2	i n</w>	1
th e</w>	3	c o u r s e</w>	1
s es s i o n</w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e</w>	1	, </w>	1
r e q u es t e d </w>	1	d u r i n g </w>	1
a </w>	1	p a r t - s es s i o n</w>	1
d e b a t e</w>	1		
o n</w>	1		
th i s </w>	2		

After several more merges

Tokenized word	Freq	Tokenized word	Freq
R es u m p t io n</w>	1	s u b j e c t </w>	1
o f </w>	2	i n</w>	1
the</w>	3	c o u r s e</w>	1
s ession</w>	1	n e x t </w>	1
. </w>	2	f e w </w>	1
Y o u </w>	1	d a y s </w>	1
h a v e</w>	1	, </w>	1
re q ues t e d </w>	1	d u r i n g </w>	1
a </w>	1	part - s ession</w>	1
d e b a t e</w>	1		
on</w>	1		
th i s </w>	2		

Does BPE learn morphology?

- BPE has **no knowledge of linguistics**
  - no notion of roots, prefixes, suffixes
- Yet some learned units *resemble meaningful parts*:
  - `re` appears in multiple words
  - `on</w>` behaves like a frequent word
  - `io` appears inside several words
- But others are **accidental fragments**:
  - `ession</w>` is not a true suffix
  - it simply reflects repeated substrings

⇒ BPE captures **frequency patterns**, not morphology

SentencePiece Unigram

Key idea:

- Alternative to BPE
- Instead of **building** subwords $\rightarrow$ **selecting** subwords

Contrast with BPE:

- BPE: merge frequent pairs step by step
- Unigram: start with many candidates and remove the least useful

Important difference:

- BPE: one segmentation at a time
- Unigram: multiple segmentations considered

Corpus representation

SentencePiece works directly on raw text.

Whitespace is treated as a symbol:

- replaced by _

Example:

_Resumption _of _the _session

Advantage:

- no language-specific tokenization needed
- can learn word-initial units like _the

Candidate subword pieces

Start with a large set of candidates

These are extracted from the corpus:

- all characters
- frequent substrings (e.g. `ion`, `re`)
- sometimes full words (e.g. `_the`, `_session`)

Key idea:

- vocabulary is initially **large and redundant**

Multiple possible segmentations

A word can be segmented in different ways:

_session

- _session
- _sess ion
- _se ss ion

Key difference with BPE:

- BPE: one segmentation
- Unigram: evaluates **all options**

How are segmentations evaluated?

Each piece gets a **score**:

- high score → useful piece
- low score → rarely useful

Example:

Piece	Score
`_session`	0.30
`_sess`	0.10
`ion`	0.25

Comparing segmentations

Segmentation 1:

$$P(\texttt{_session}) = 0.30$$

Segmentation 2:

$$P(\texttt{_sess ion}) = 0.10 \times 0.25 = 0.025$$

Interpretation:

- the model prefers `_session`
- it is a more efficient explanation

Where do the scores come from?

Scores are learned from the corpus:

- try segmenting all words
- count how often pieces are used
- useful pieces get higher scores

Intuition:

- frequent + useful $\rightarrow$ high score
- rare + useless $\rightarrow$ low score

Pruning the vocabulary

Training proceeds in cycles:

- evaluate all pieces
- remove the least useful ones
- re-evaluate remaining pieces

Goal:

- reduce to a fixed vocabulary size

Key idea:

- start big $\rightarrow$ gradually simplify

Final vocabulary

Result:

- frequent words kept intact:
  - `_the`, `_of`
- less frequent words split:
  - `_request ed`

Observation:

- mix of full words + subword units

BPE vs SentencePiece Unigram

- BPE:
  - builds tokens step by step
  - bottom-up
- SentencePiece Unigram:
  - starts with many candidates
  - removes the least useful
  - top-down

BPE constructs units
Unigram selects units

Hands-on notebooks (Kaggle)

Subwording + RNN with SentencePiece:

mtat26-subwording-transformers

(Installation commands, training commands, and plotting are documented inside the notebooks.)

Subwording experiments: what we tested

We compare:

- **Separate SentencePiece models** (src and tgt each trained on its own corpus)
- **Joint vocabulary** (single SentencePiece model trained on concatenated corpora)

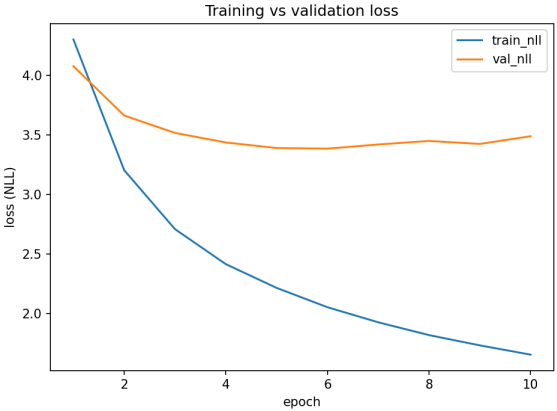
We monitor:

- training vs validation loss (overfitting),
- BLEU and chrF on dev,
- comparison with earlier RNN/GRU/LSTM/biLSTM(+att) baselines.

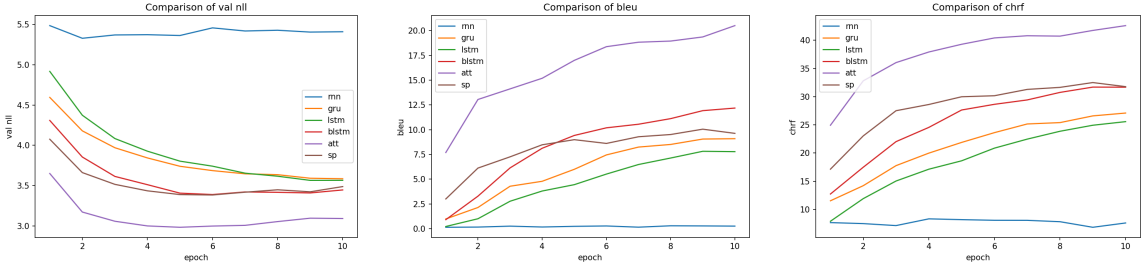
Results: separate SentencePiece models

Key observation from the notes:

- The model still tends to **overfit** after a few epochs.
- In this small-data setting, subwording does **not necessarily improve** BLEU/chrF.



Comparing systems (NLL / BLEU / chrF)



Comparison with earlier RNN-based systems

Joint vocabulary: why it often helps

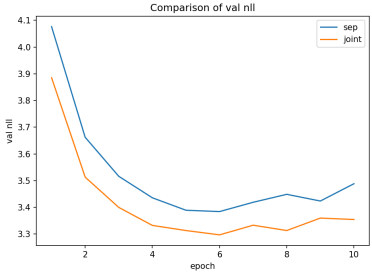
Joint subword vocabularies are common in MT and multilingual settings:

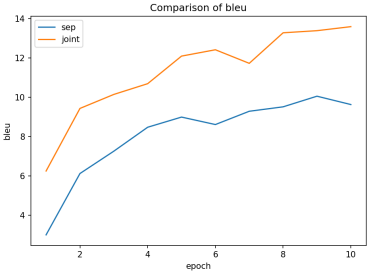
- encourages sharing of subword units across languages,
- helps align cognates/morphemes and shared function words,
- reduces total vocabulary size (one model instead of two).

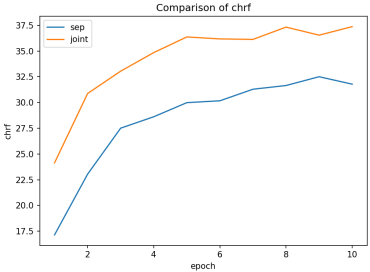
Workflow:

1. concatenate corpora,
2. train SentencePiece on the joint corpus,
3. train NMT with the *same* model for src and tgt.

Results: separate vs joint SentencePiece







Joint SentencePiece vs separate: joint tends to score consistently better in the notes.

Transformers

Motivation: beyond recurrent models

RNNs process tokens **sequentially**:

- limited parallelisation $\Rightarrow$ slower training,
- long-range dependencies can be difficult to capture.

Transformers ((Vaswani et al. 2017)) remove recurrence:

- all tokens processed **in parallel** via attention,
- better scalability and strong MT quality improvements.

Core idea: attention is all you need

Each token directly attends to all other tokens in the sentence, independently of distance.

This is **self-attention**: tokens build contextual representations by selectively focusing on other tokens.

Self-attention: what it does

Given a sentence (after subwording + embeddings + positional encoding), self-attention updates each token representation using information from:

- itself,
- all other tokens in the same sentence.

Example sentence:

She eats apples .

Queries, keys, values (Q, K, V)

For each token we derive:

- **Query** (what I am looking for),
- **Key** (what I offer / how I can be matched),
- **Value** (what information I contribute).

For token `eats`:

- compare $Q_{\texttt{eats}}$ with K of all tokens (`she`, `eats`, `apples`, punctuation),
- obtain attention weights,
- form a weighted sum of the corresponding V vectors.

Attention weights → contextual representations

Self-attention produces **contextualised** vectors:

- `eats` can incorporate information about its subject/object,
- `apples` can attend to `eats` to encode its role as object,
- punctuation typically receives low attention in content heads.

So the representation of a token depends on **the whole sentence**.

Why self-attention matters in MT: pronouns

Source language ambiguity can force explicit choices in the target language.

English pronoun `it` can be ambiguous, but French requires grammatical gender:

- *il* (masc.) vs *elle* (fem.)

Self-attention can connect `it` to the most plausible antecedent in context.

Example from Transformer explanations (in notes)

The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.

Figure: Two sentences requiring pronoun resolution for correct French translation.

Self-attention visualisation for `it`

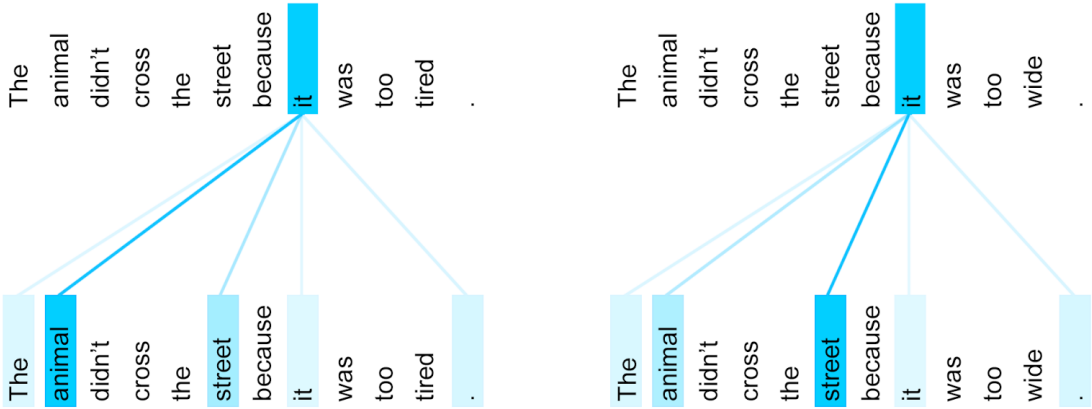


Figure: Attention patterns: `it` attends to different antecedents depending on context.

Multi-head attention

Transformers compute multiple attention distributions in parallel:

- each head has its own Q/K/V projections,
- heads can specialise (agreement, modifiers, long-distance links, ...),
- head outputs are concatenated and linearly transformed.

Breadth: multiple relational views at the same layer.

Multiple layers: progressive refinement

Encoders/decoders are stacks of identical layers.

- lower layers capture more surface/local associations,
- higher layers integrate broader context and semantics,
- ambiguity can be resolved progressively across layers.

Depth: repeated transformation and integration.

Heads + layers: complementary roles

- **Multi-head attention** increases *breadth*: diverse relations in parallel.
- **Stacked layers** increase *depth*: refine and combine evidence over layers.

Both are essential for strong contextual modelling in MT.

Transformer decoder: 3 components per layer

Each decoder layer contains:

1. **Masked self-attention** (only attend to previous target tokens),
2. **Cross-attention** (attend to encoder outputs),
3. **Feed-forward network** (transform combined information).

Stacking decoder layers integrates target context with source context progressively.

Autoregressive decoding (and beam search)

At inference time, Transformers generate tokens **sequentially**:

- each next token conditions on all previous target tokens (via masked self-attention),
- and on the full source sentence (via cross-attention).

Unlike RNNs, there is no single recurrent hidden state; attention recomputes over the available context each step.

Beam search explores multiple hypotheses during decoding.

Why Transformers work well for MT

Transformers:

- avoid recurrence bottlenecks,
- enable parallel training,
- model long-distance dependencies effectively,
- work naturally with subword vocabularies,
- scale well and deliver strong MT quality.

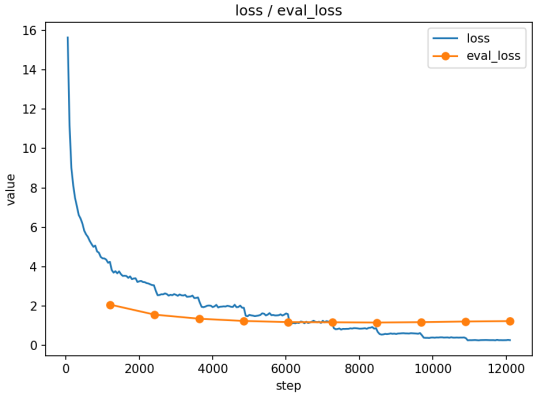
They are now the dominant MT architecture and underpin many pretrained models.

Transformer training: what to look for

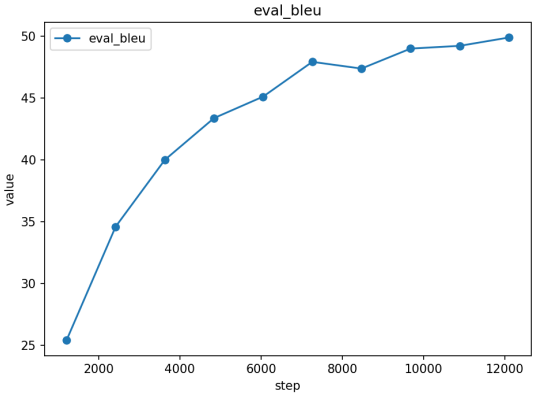
In the hands-on notebook:

- **Transformers (bilingual):** mtat26-transformers
- **Multilingual Transformers + zero-shot:** mtat26-multilingualtransformers
- validation is computed every fixed number of steps (not only per epoch),
- early epochs can show higher train loss than dev loss,
- BLEU can reach much higher values than in the earlier RNN experiments.

Transformer results



Validation loss over epochs.



BLEU score over epochs.

Multilingual Neural Machine Translation

Why multilingual MT?

Training one bilingual model per language pair is often impractical.
MNMT trains **one model** on multiple directions:

- parameter sharing across languages,
- shared subword vocabularies,
- explicit conditioning on the target language.

It raises questions about **cross-lingual transfer** vs **negative transfer**.

Target language tags

A common method: prepend a target-language tag to the source sentence, e.g.

<2nl> She eats apples .

The tag conditions the decoder on the intended output language, enabling one model to serve multiple directions.

Shared subwords in multilingual MT

Shared vocabularies:

- encourage alignment of cognates/morphemes/function words,
- reduce model complexity (one vocab),
- support transfer (especially for related languages).

Zero-shot translation

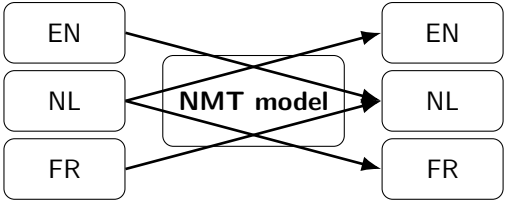
Zero-shot can emerge when a multilingual model learns a shared space:

- trained on some directions (e.g. EN↔NL and FR↔NL),
- can sometimes translate EN↔FR without direct EN–FR supervision.

This relies on **parameter sharing** and **cross-lingual generalisation**.

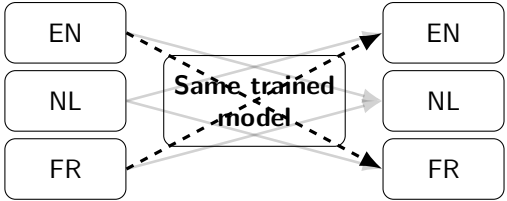
Zero-shot translation in MNMT (training vs inference)

Phase 1: multilingual training



(a) Training directions.

Phase 2: zero-shot inference



(b) Add EN↔FR zero-shot.

Model capacity and data imbalance

Challenges:

- high-resource languages dominate training,
- low-resource directions may degrade,
- interference can cause **negative transfer**.

Mitigations:

- temperature-based sampling / upsampling,
- language-specific adapters / routing,
- increase model capacity.

Evaluation in multilingual MT

Evaluation is more complex than bilingual MT:

- performance varies by language and direction,
- improvements in one pair can coincide with drops in another,
- report per-direction + macro-averages where appropriate.

Hands-on MNMT: pipeline overview

In the multilingual notebook we:

1. add target-language tags to source sentences,
2. concatenate all directions into one multilingual training/dev set,
3. train a **joint** SentencePiece model,
4. train a multilingual Transformer,
5. test a **zero-shot** direction by using a new tag at inference.

Multilingual Transformer results

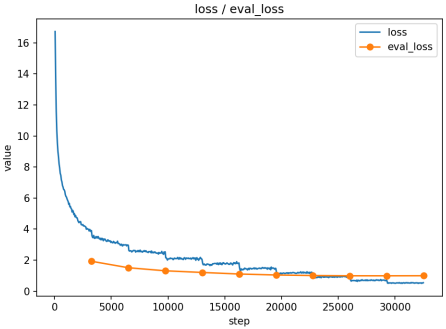


Figure: Validation loss over epochs.

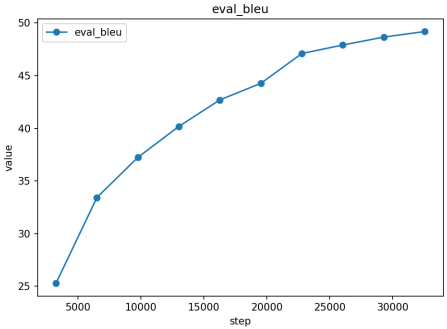


Figure: BLEU score over epochs (combined dev set).

Note: BLEU here aggregates multiple directions, so it is not strictly comparable to bilingual BLEU.

Zero-shot test: what to expect

When testing EN→FR zero-shot on this small setup:

- translations are typically **not very good**,
- but you may observe **some** learned ability to produce French,
- the limiting factor is the **small scale** of the Tatoeba data and limited multilingual signal.

Takeaway: zero-shot is possible in principle, but quality strongly depends on data scale, coverage, and model capacity.